

- **ReactJS Concept Guide: 25+ Questions & Answers**

1. What is ReactJS?

Answer:

ReactJS is a **JavaScript library for building user interfaces**. It is maintained by Facebook and focuses on creating reusable UI components for **single-page applications (SPAs)**. React uses a **virtual DOM** for efficient rendering.

2. What is the Virtual DOM?

Answer:

The **Virtual DOM** is a lightweight copy of the actual DOM. React updates the Virtual DOM first and then **diffs** it with the real DOM to apply **minimal changes**, improving performance.

3. What are React components?

Answer:

Components are **reusable building blocks** of a React app.

- **Functional Components:** Use functions and hooks.
 - **Class Components:** Use ES6 classes and lifecycle methods.
-

4. What is JSX?

Answer:

JSX (JavaScript XML) is a **syntax extension** that allows you to write **HTML-like code inside JavaScript**. React converts JSX into `React.createElement()` calls.

5. What are props in React?

Answer:

Props (properties) are **read-only inputs** passed from parent to child components. They help make components **reusable**.

6. What is state in React?

Answer:

State is **mutable data managed within a component**. Changing the state triggers a **re-render** of the component.

7. Difference between State and Props Answer:

Feature	State	Props
Mutable	Yes	No
Managed by	Component itself	Parent component
Purpose	Dynamic data	Pass data

8. What are React Hooks?

Answer:

Hooks are **functions that let functional components use state and lifecycle features**. Examples:

- `useState()`
 - `useEffect()`
 - `useContext()`
-

9. Explain useState Hook

Answer:

`useState()` lets you **add state to functional components**.

```
const [count, setCount] = useState(0);
```

- `count` = current value
 - `setCount` = function to update value
-

10. Explain useEffect Hook

Answer:

useEffect() manages **side effects** like API calls, subscriptions, or DOM updates. It replaces most **lifecycle methods**.

```
useEffect(() => { console.log('Component  
mounted or updated');  
}, [dependencies]);
```

11. What are React lifecycle methods?

Answer:

Lifecycle methods exist in **class components**:

- componentDidMount() – Runs after component mounts
 - componentDidUpdate() – Runs after update
 - componentWillUnmount() – Runs before unmount
-

12. What is a controlled component?

Answer:

A controlled component has **form input values controlled by React state**.

```
<input value={stateValue} onChange={handleChange} />
```

13. What is an uncontrolled component?

Answer:

Uncontrolled components manage their **own internal state**, using **refs** to access values.

14. What is React Router?

Answer:

React Router is a library for **client-side routing**, allowing **SPA navigation** without page reloads.

- <BrowserRouter>

- <Route>
- <Link>

15. What is the difference between React and Angular?

Answer:

Feature	React	Angular
Type	Library	Framework
Data Binding	One-way	Two-way
DOM	Virtual DOM	Real DOM
Learning Curve	Moderate	Steep

16. What is Redux?

Answer:

Redux is a **state management library** for React apps. It centralizes state in a **store**, making it predictable.

17. What are React keys?

Answer:

Keys help React **identify which items in a list have changed** for efficient re-rendering. They must be **unique and stable**.

18. What is context in React?

Answer:

Context allows **passing data through the component tree without props drilling**.

```
const ThemeContext = React.createContext();
```

19. What are higher-order components (HOC)?

Answer:

HOCs are **functions that take a component and return a new enhanced component.**

```
const EnhancedComponent = withHOC(MyComponent);
```

20. What is React Fragment?

Answer:

Fragments let you **group multiple elements without adding extra DOM nodes:**

```
<React.Fragment>
  <Child1 />
  <Child2 />
</React.Fragment>
```

21. How to handle events in React?

Answer:

React uses **camelCase events** and **functions instead of strings:**

```
<button onClick={handleClick}>Click Me</button>
```

22. Difference between class and functional components Answer:

Feature	Class	Functional
Syntax	ES6 class	Function
State	Yes	Yes (with hooks)
Lifecycle	Yes	Use useEffect

23. What is lazy loading in React?

Answer:

Lazy loading loads **components only when needed**, improving performance:

```
const LazyComponent = React.lazy(() => import('./Component'));
```

24. What is React Suspense?

Answer:

Suspense is used with **lazy loading** to show a fallback UI until the component **finishes loading**:

```
<Suspense fallback={<div>Loading...</div>}>
```

```
  <LazyComponent />
```

```
</Suspense>
```

25. How to optimize React performance?

Answer:

- Use **React.memo** to memoize components
 - Use **useCallback** and **useMemo** hooks
 - Split code using **lazy loading**
 - Avoid unnecessary state updates
-

26. Difference between virtual DOM and real DOM

Answer:

- **Virtual DOM:** Lightweight copy, faster updates, React uses diffing
- **Real DOM:** Browser DOM, slower for updates

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like Redux.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
```

```
<ThemeContext.Provider value={theme}>
```

```
  <ChildComponent />
```

```
</ThemeContext.Provider>
```

29. Difference between useEffect and useLayoutEffect

Answer:

Hook	Timing	Use Case
useEffect	After DOM paint	API calls, side effects
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?

Answer:

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?

Answer:

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?

Answer:

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?

Answer:

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?

Answer:

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?

Answer:

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?

Answer:

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?

Answer:

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**. Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.
`<DataFetcher render={data => <Child data={data} /> } />`

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
 - `shouldComponentUpdate()` in class components
 - Memoize functions/values using `useCallback` / `useMemo`
-

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
 - **SSR:** Server renders HTML for faster initial load and SEO (Next.js)
-

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`

- Store results in state
- Handle loading/error states

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
- **Uncontrolled:** Input state is managed by DOM, accessed via ref

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions) and generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like Redux.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
<ThemeContext.Provider value={theme}>
  <ChildComponent />
</ThemeContext.Provider>
```

29. Difference between useEffect and useLayoutEffect

Answer:

Hook	Timing	Use Case
useEffect	After DOM paint	API calls, side effects
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?**Answer:**

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?**Answer:**

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?**Answer:**

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?**Answer:**

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?**Answer:**

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?**Answer:**

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?**Answer:**

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?**Answer:**

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**.

Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.

```
<DataFetcher render={data => <Child data={data} />} />
```

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
 - `shouldComponentUpdate()` in class components
 - Memoize functions/values using `useCallback` / `useMemo`
-

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
- **SSR:** Server renders HTML for faster initial load and SEO (Next.js)

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`
 - Store results in state
 - Handle loading/error states
-

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
 - **Uncontrolled:** Input state is managed by DOM, accessed via `ref`
-

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions) and generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like `Redux`.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
```

```
<ThemeContext.Provider value={theme}>
```

```
  <ChildComponent />
```

```
</ThemeContext.Provider>
```

29. Difference between `useEffect` and `useLayoutEffect`

Answer:

Hook	Timing	Use Case
<code>useEffect</code>	After DOM paint	API calls, side effects

Hook	Timing	Use Case
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.
`export default React.memo(MyComponent);`

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?

Answer:

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.
`ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));`

34. What is error boundary in React?

Answer:

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.
`componentDidCatch(error, info) { ... }`

35. What is the difference between controlled and uncontrolled components in forms?

Answer:

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?

Answer:

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?

Answer:

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?

Answer:

- Controlled components with `useState`

- Handle input changes via onChange
- Submit via onSubmit handler

39. What is lazy loading in React?

Answer:

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with React.lazy() and Suspense.

40. What is the difference between React and React Native?

Answer:

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
- **Deep Rendering:** Renders component with children (integration test).

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**. Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.
`<DataFetcher render={data => <Child data={data} /> } />`

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
- **State:** Mutable, managed within component

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
- `shouldComponentUpdate()` in class components
- Memoize functions/values using `useCallback` / `useMemo`

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
- **SSR:** Server renders HTML for faster initial load and SEO (Next.js)

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`
- Store results in state
- Handle loading/error states

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
- **Uncontrolled:** Input state is managed by DOM, accessed via `ref`

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions)** and **generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like Redux.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
<ThemeContext.Provider value={theme}>
  <ChildComponent />
</ThemeContext.Provider>
```

29. Difference between useEffect and useLayoutEffect

Answer:

Hook	Timing	Use Case
useEffect	After DOM paint	API calls, side effects
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?

Answer:

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?

Answer:

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?

Answer:

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?

Answer:

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?

Answer:

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?

Answer:

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?

Answer:

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?

Answer:

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**. Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.

```
<DataFetcher render={data => <Child data={data} /> } />
```

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- React.memo() for functional components
 - shouldComponentUpdate() in class components
 - Memoize functions/values using useCallback / useMemo
-

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<Redirect>	<Navigate>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
 - **SSR:** Server renders HTML for faster initial load and SEO (Next.js)
-

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using React.lazy().

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`
- Store results in state
- Handle loading/error states

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
- **Uncontrolled:** Input state is managed by DOM, accessed via `ref`

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions) and generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like `Redux`.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
<ThemeContext.Provider value={theme}>
  <ChildComponent />
</ThemeContext.Provider>
```

29. Difference between `useEffect` and `useLayoutEffect`

Answer:

Hook	Timing	Use Case
<code>useEffect</code>	After DOM paint	API calls, side effects
<code>useLayoutEffect</code>	Before DOM paint	Measure DOM, update layout synchronously

30. What is `React.memo`?

Answer:

`React.memo` **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between `useCallback` and `useMemo`?

Answer:

Hook	Purpose
<code>useCallback</code>	Memoizes functions
<code>useMemo</code>	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?

Answer:

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?

Answer:

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?

Answer:

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?

Answer:

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?

Answer:

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?

Answer:

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?

Answer:

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?

Answer:

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**.

Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.

```
<DataFetcher render={data => <Child data={data} />} />
```

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
 - `shouldComponentUpdate()` in class components
 - Memoize functions/values using `useCallback` / `useMemo`
-

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)

- **SSR:** Server renders HTML for faster initial load and SEO (Next.js)

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`
- Store results in state
- Handle loading/error states

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
- **Uncontrolled:** Input state is managed by DOM, accessed via `ref`

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions)** and **generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like `Redux`.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
<ThemeContext.Provider value={theme}>
  <ChildComponent />
</ThemeContext.Provider>
```

29. Difference between `useEffect` and `useLayoutEffect`

Answer:

Hook	Timing	Use Case
useEffect	After DOM paint	API calls, side effects
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?

Answer:

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?

Answer:

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?

Answer:

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?

Answer:

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?

Answer:

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?

Answer:

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?

Answer:

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?

Answer:

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**. Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.
`<DataFetcher render={data => <Child data={data} /> } />`

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
- `shouldComponentUpdate()` in class components
- Memoize functions/values using `useCallback` / `useMemo`

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
- **SSR:** Server renders HTML for faster initial load and SEO (Next.js)

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`
- Store results in state
- Handle loading/error states

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
- **Uncontrolled:** Input state is managed by DOM, accessed via `ref`

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions)** and **generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like Redux.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
```

```
<ThemeContext.Provider value={theme}>
```

```
  <ChildComponent />
```

```
</ThemeContext.Provider>
```

29. Difference between useEffect and useLayoutEffect

Answer:

Hook	Timing	Use Case
useEffect	After DOM paint	API calls, side effects
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?

Answer:

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?

Answer:

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?

Answer:

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?

Answer:

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?

Answer:

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?

Answer:

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?

Answer:

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?

Answer:

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**. Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.
`<DataFetcher render={data => <Child data={data} /> } />`

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
 - `shouldComponentUpdate()` in class components
 - Memoize functions/values using `useCallback` / `useMemo`
-

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
 - **SSR:** Server renders HTML for faster initial load and SEO (Next.js)
-

51. What is hydration in React?

Answer:

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?

Answer:

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?

Answer:

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?

Answer:

- Use `fetch()` or `Axios` inside `useEffect`

- Store results in state
- Handle loading/error states

55. What are controlled vs uncontrolled components?

Answer:

- **Controlled:** Input state is managed by React
- **Uncontrolled:** Input state is managed by DOM, accessed via ref

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions) and generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?

ReactJS Concept Guide: Next 25 Questions & Answers

27. What is prop drilling?

Answer:

Prop drilling happens when **data is passed through multiple levels of components** just to reach a deeply nested component. It can make code harder to maintain.

Solution: Use **Context API** or **state management libraries** like Redux.

28. What is the Context API?

Answer:

The **Context API** allows you to **share data across components** without manually passing props at every level.

```
const ThemeContext = React.createContext();
<ThemeContext.Provider value={theme}>
  <ChildComponent />
</ThemeContext.Provider>
```

29. Difference between useEffect and useLayoutEffect

Answer:

Hook	Timing	Use Case
useEffect	After DOM paint	API calls, side effects
useLayoutEffect	Before DOM paint	Measure DOM, update layout synchronously

30. What is React.memo?

Answer:

React.memo **memoizes functional components**, preventing unnecessary re-renders if props haven't changed.

```
export default React.memo(MyComponent);
```

31. What is the difference between useCallback and useMemo?

Answer:

Hook	Purpose
useCallback	Memoizes functions
useMemo	Memoizes values/results of computations

32. What is Reconciliation in React?

Answer:

Reconciliation is React's process of **updating the DOM efficiently** by comparing the Virtual DOM with the real DOM and applying minimal changes.

33. What are portals in React?**Answer:**

Portals allow rendering a child component **outside the parent DOM hierarchy**. Useful for modals, tooltips.

```
ReactDOM.createPortal(<Modal />, document.getElementById('modal-root'));
```

34. What is error boundary in React?**Answer:**

Error boundaries **catch JavaScript errors** in child components, preventing the entire app from crashing. Only **class components** can be error boundaries.

```
componentDidCatch(error, info) { ... }
```

35. What is the difference between controlled and uncontrolled components in forms?**Answer:**

- **Controlled:** React manages the value (state).
 - **Uncontrolled:** DOM manages the value, accessed via ref.
-

36. What are fragments used for?**Answer:**

Fragments let you **wrap multiple elements without adding extra nodes** to the DOM. Use `<React.Fragment>` or `<>`.

37. What is the difference between SPA and MPA?**Answer:**

- **SPA (Single Page App):** Loads one HTML page, updates content dynamically (React apps).
 - **MPA (Multi Page App):** Loads multiple HTML pages, full page reloads.
-

38. How to handle forms in React?**Answer:**

- Controlled components with `useState`
 - Handle input changes via `onChange`
 - Submit via `onSubmit` handler
-

39. What is lazy loading in React?**Answer:**

Lazy loading **defers loading components** until needed, reducing initial bundle size. Works with `React.lazy()` and `Suspense`.

40. What is the difference between React and React Native?**Answer:**

Feature	React	React Native
Platform	Web	Mobile (iOS & Android)
Rendering	HTML	Native UI components
Styling	CSS	StyleSheet objects

41. What is the difference between shallow and deep rendering in testing?

Answer:

- **Shallow Rendering:** Renders only the component, not children (unit test).
 - **Deep Rendering:** Renders component with children (integration test).
-

42. What is the use of key in lists?

Answer:

Keys help React **identify elements uniquely** to optimize rendering of lists and prevent unnecessary re-renders.

43. What is a Higher-Order Component (HOC)?

Answer:

HOCs are functions that **take a component and return an enhanced component**.

Example: authentication wrapper.

44. What are render props?

Answer:

A render prop is a **function prop that a component uses to know what to render**.

```
<DataFetcher render={data => <Child data={data} />} />
```

45. What is the difference between props vs state?

Answer:

- **Props:** Read-only, passed from parent
 - **State:** Mutable, managed within component
-

46. What is reconciliation in React?

Answer:

The process of **updating the DOM efficiently** by comparing old and new virtual DOM trees and applying only necessary changes.

47. What are synthetic events in React?

Answer:

React creates **Synthetic Events** to wrap native events, providing **cross-browser compatibility**.

48. How do you prevent re-rendering in React?

Answer:

- `React.memo()` for functional components
 - `shouldComponentUpdate()` in class components
 - Memoize functions/values using `useCallback` / `useMemo`
-

49. What is the difference between React Router v5 and v6?

Answer:

Feature	v5	v6
Route nesting	Implicit	Explicit
Switch	Yes	Replaced by Routes
Redirect	<code><Redirect></code>	<code><Navigate></code>

50. What is the difference between server-side rendering (SSR) and client-side rendering (CSR) in React?

Answer:

- **CSR:** Browser renders React components (SPA)
- **SSR:** Server renders HTML for faster initial load and SEO (Next.js)

51. What is hydration in React?**Answer:**

Hydration **attaches React's event listeners to server-rendered HTML** in SSR apps.

52. What are portals used for in React?**Answer:**

Portals let you **render a component outside its parent DOM** (modals, tooltips) while keeping React's state management intact.

53. What is suspense fallback in React?**Answer:**

The **fallback UI** shown while a component is **loading asynchronously** using `React.lazy()`.

54. How to fetch data in React?**Answer:**

- Use `fetch()` or `Axios` inside `useEffect`
 - Store results in state
 - Handle loading/error states
-

55. What are controlled vs uncontrolled components?**Answer:**

- **Controlled:** Input state is managed by React
 - **Uncontrolled:** Input state is managed by DOM, accessed via `ref`
-

This completes the **next 25 questions**, bringing your guide to **50 comprehensive React questions**.

If you want, I can **combine both sets (50 questions)** and **generate a polished PDF** with **code formatting, headings, and table of contents** so it's ready to download and use as a study guide.

Do you want me to generate that PDF now?